Pre-entrega 2

Orquestación Multi-Agente Distribuida

Guardián de Gastos · Textil Nordelta

José Abrahan Bonilla

AI Automation Avanzado · Módulo 2 · Coderhouse

1. Resumen de la arquitectura

El agente del Módulo 1 se dividió en un sistema Manager-Worker. El Manager recibe la rendición por chat, la clasifica con un Router y delega en dos especialistas, cada uno en su propio sub-workflow:

- **Worker 1 · Validador:** decide si el gasto cumple la política consultando la planilla (AI Agent con la tool consultar_politica_de_gastos).
- **Worker 2 · Redactor:** convierte esa decisión en un mensaje para el empleado (Basic LLM Chain, sin herramientas).

Los pedidos que no son rendiciones se resuelven sin llamar a los Workers: con una respuesta fija o escalando a un supervisor humano.

2. Workflow Manager

Flujo principal: Chat Trigger → Router (AI Agent con taxonomía cerrada) → Switch de enrutamiento. La rama Rendición encadena los dos Workers mediante nodos Execute Workflow, cada uno precedido por un nodo Set que limpia los datos.

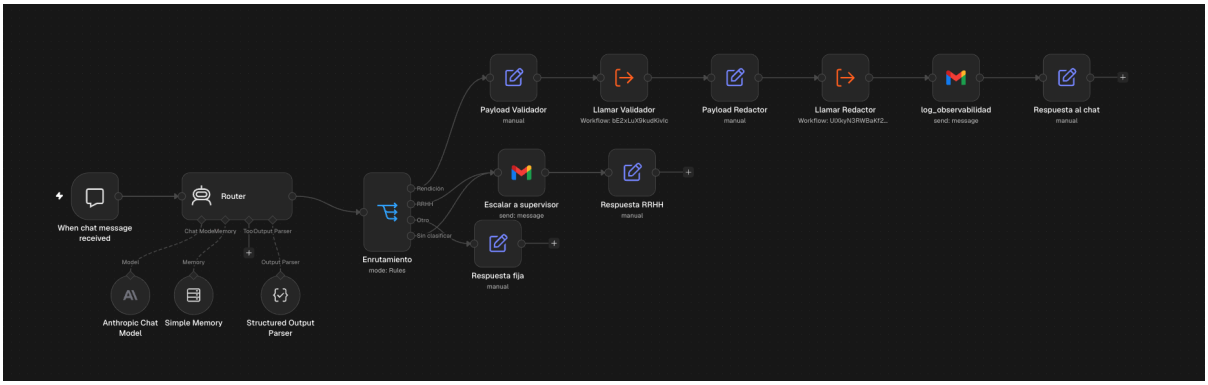


Figura 1. Lienzo completo del Workflow Manager, con los nodos Execute Workflow (Llamar Validador y Llamar Redactor).

3. Workers (sub-workflows)

Cada Worker vive en su propio lienzo, empieza con el nodo Execute Workflow Trigger (When Executed by Another Workflow) y termina en un nodo de salida que devuelve siempre el mismo contrato JSON.

3.1 Worker 1 · Validador

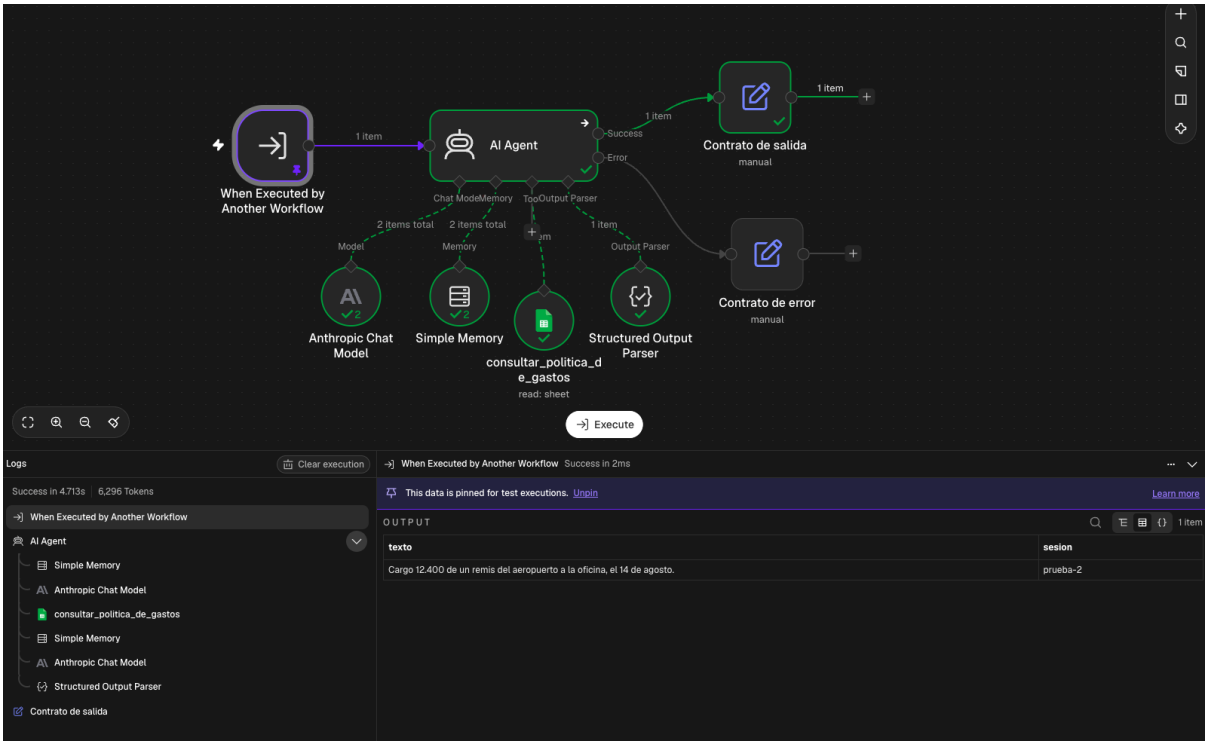


Figura 2. Lienzo del Worker 1: trigger, AI Agent con modelo, memoria, tool y Output Parser, y nodos de salida.

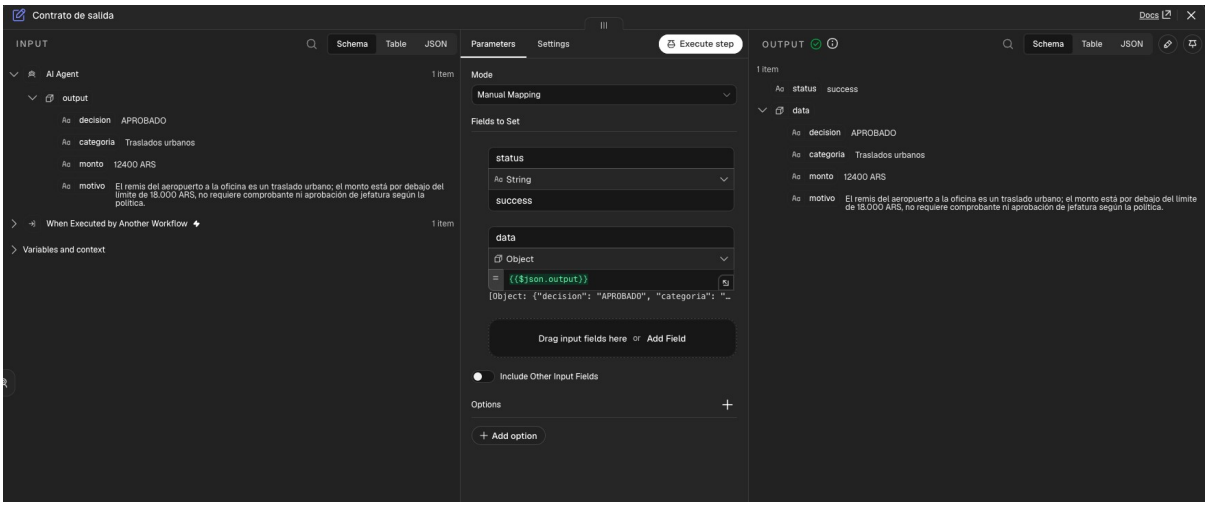


Figura 3. Salida del Worker 1 (Contrato de salida): status "success" y data con decision, categoria, monto y motivo.

3.2 Worker 2 · Redactor

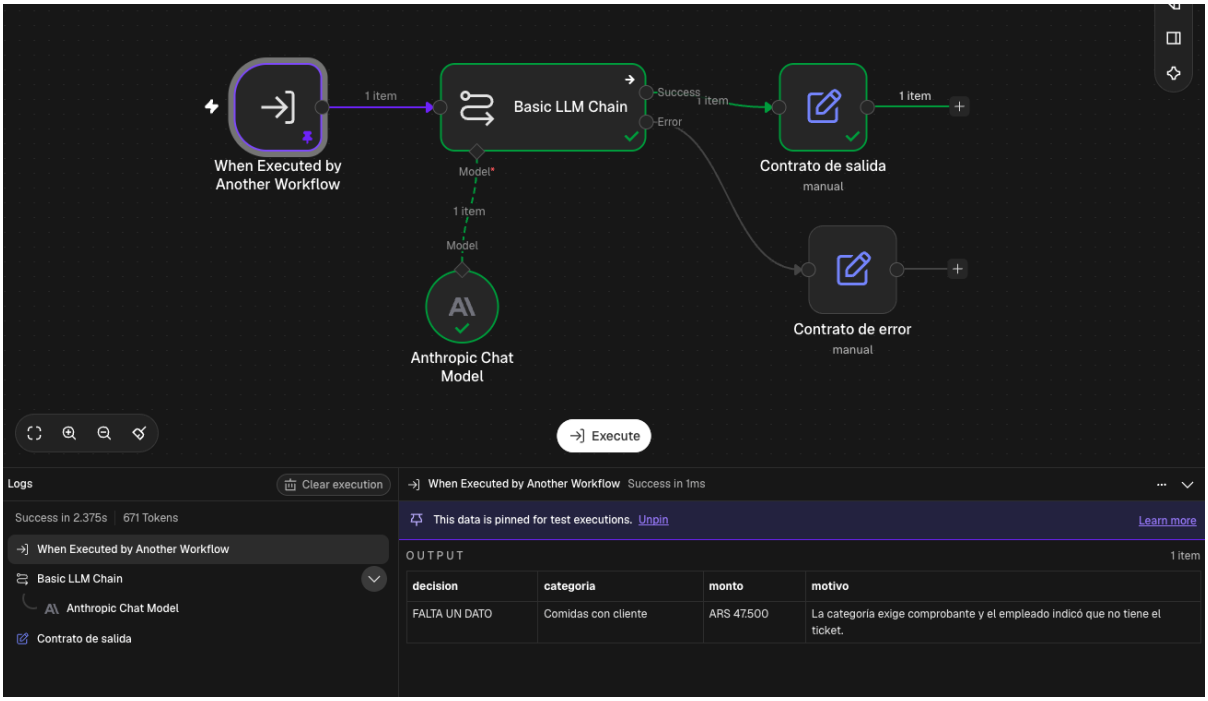


Figura 4. Lienzo del Worker 2: trigger, Basic LLM Chain y nodos de salida.

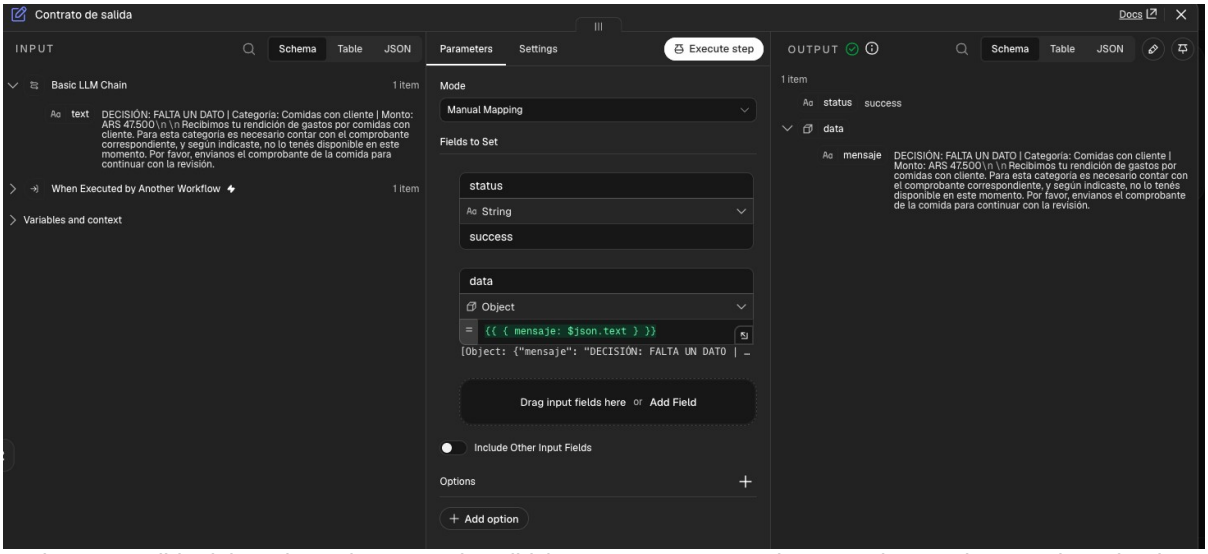


Figura 5. Salida del Worker 2 (Contrato de salida): status "success" y data con el mensaje para el empleado.

4. Handoff: nodo Execute Workflow

El Manager delega con el nodo Execute Workflow configurado así:

- **Workflow:** Worker 1 - Validador, elegido desde la base de datos de n8n.
- **Workflow Inputs:** solo texto y sesion, que llegan limpios desde el nodo Set "Payload Validador".
- **Wait For Sub-Workflow Completion (Wait for child to finish):** activado, para que el Manager espere el JSON del Worker antes de seguir.
- **Timeout:** cada Worker tiene activado Timeout Workflow en 1 minuto, para que una API lenta no deje al Manager esperando indefinidamente.

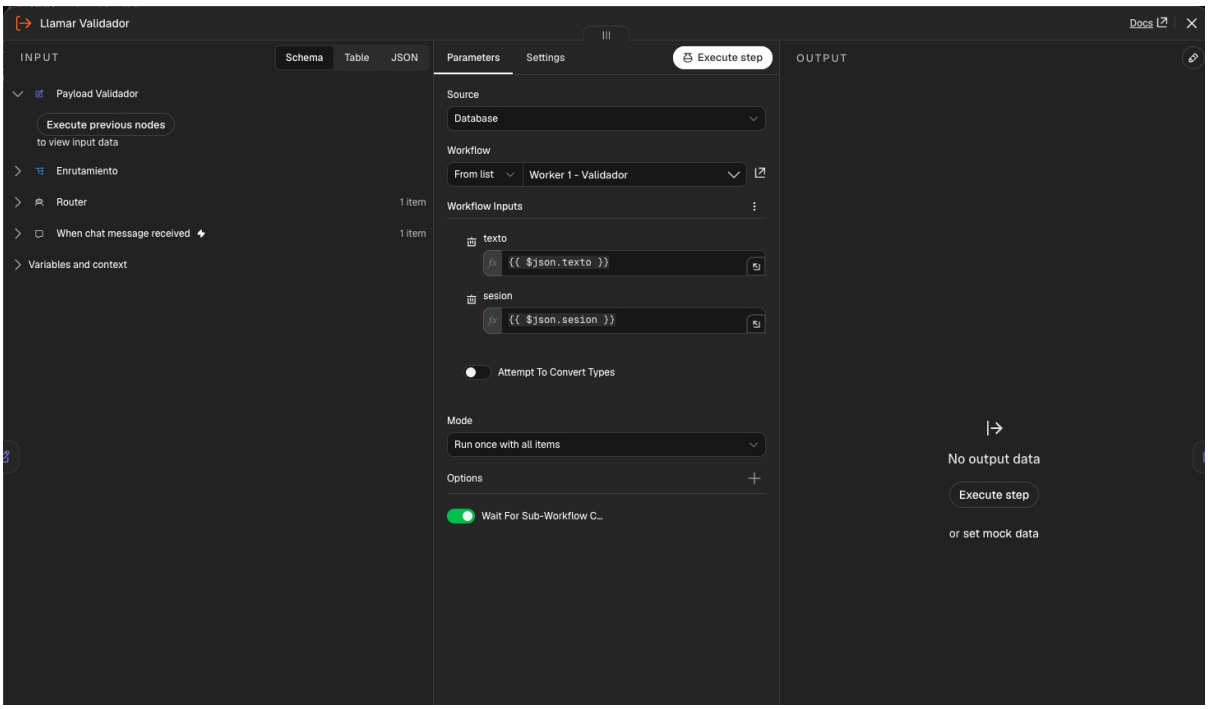


Figura 6. Configuración del nodo Llamar Validador: datos enviados al Worker y Wait For Sub-Workflow Completion (Wait for child to finish) activado.

4.1 Timeout de los Workers

Ambos Workers tienen activado Timeout Workflow en 1 minuto: si un Worker se demora, la ejecución se corta y no deja al Manager esperando indefinidamente.

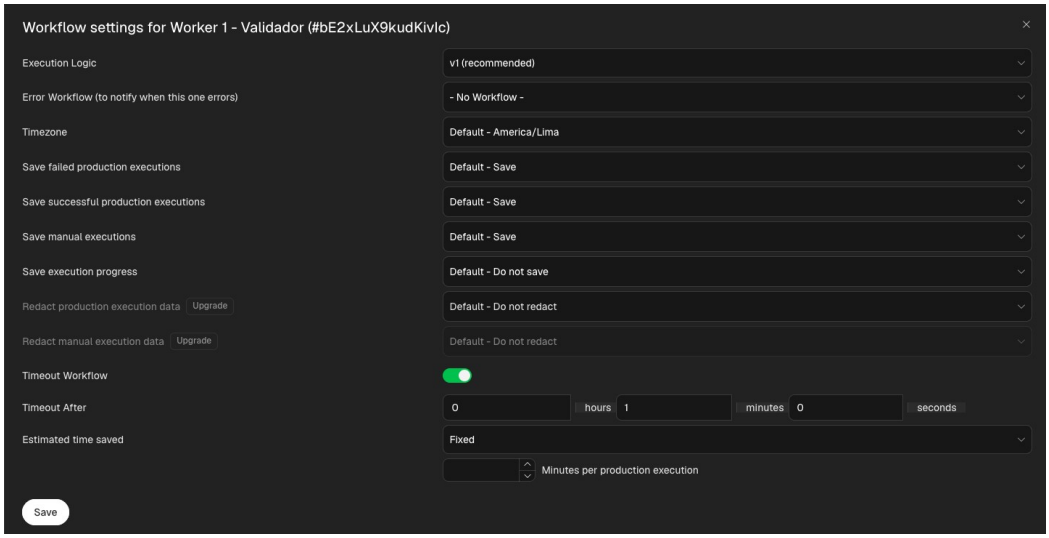


Figura 7. Timeout del Worker 1 · Validador: Timeout Workflow activado en 1 minuto.

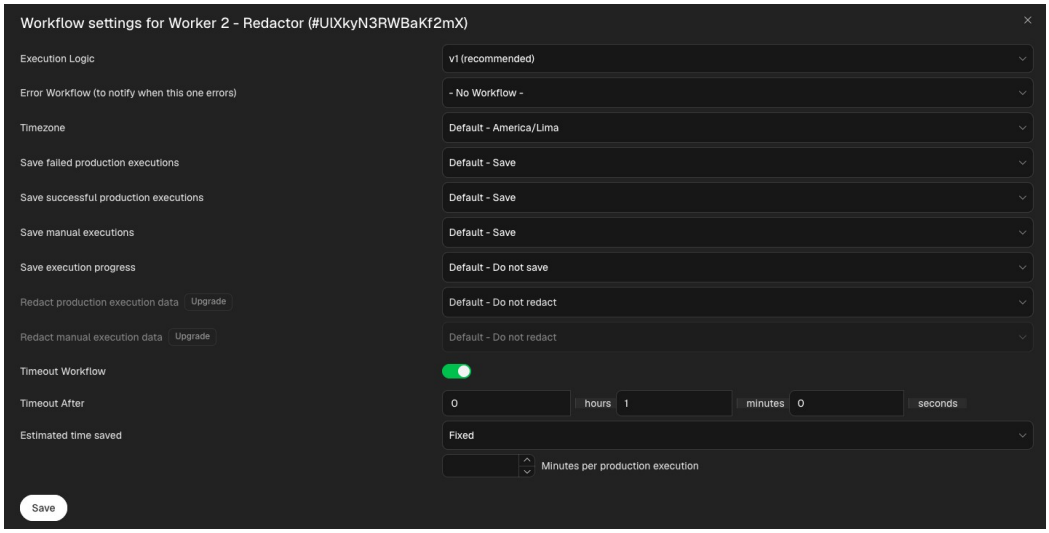


Figura 8. Timeout del Worker 2 · Redactor: Timeout Workflow activado en 1 minuto.

5. Ejecución de punta a punta

Prueba: **"Compré 3 tazas con el logo para regalarle al proveedor nuevo, 21.000."** El Router la clasifica como RENDICION, el Validador deduce Representacion y escala por requerir aprobación de jefatura, y el Redactor arma el mensaje.

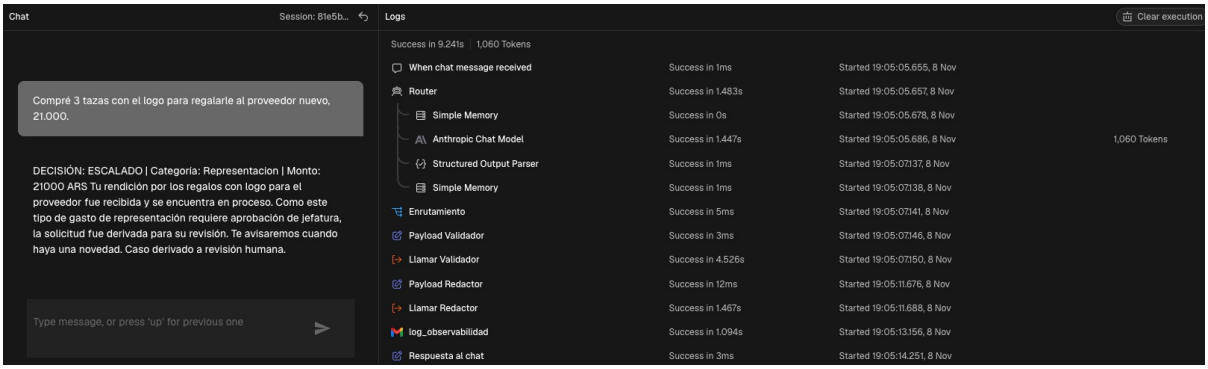


Figura 9. Panel de ejecución del Manager: Router, Switch, Validador, Redactor, log y respuesta, todos en verde.



Figura 10. Log de trazabilidad en Gmail: qué Worker se invocó, con qué estado y qué respondió.

Prueba de memoria en dos mensajes: el empleado primero envía solo el monto y luego el concepto. El sistema recuerda el monto y pide el siguiente dato que exige la política.

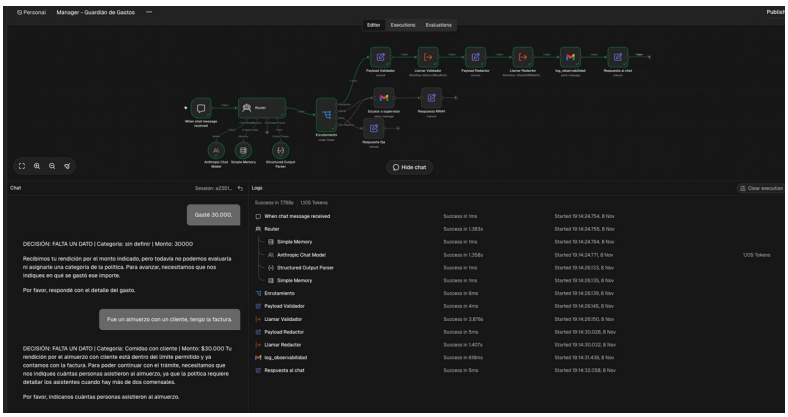


Figura 11. Conversación en dos turnos: FALTA UN DATO (concepto) y luego FALTA UN DATO (cantidad de comensales).

6. Rutas de error de los Workers

Cada Worker tiene configurado On Error → Continue (using error output). Si algo falla, no muere en silencio: devuelve un JSON de contingencia con status "error" y el Manager sigue funcionando.

6.1 Worker 1 · Validador (forzando Max Iterations = 1)

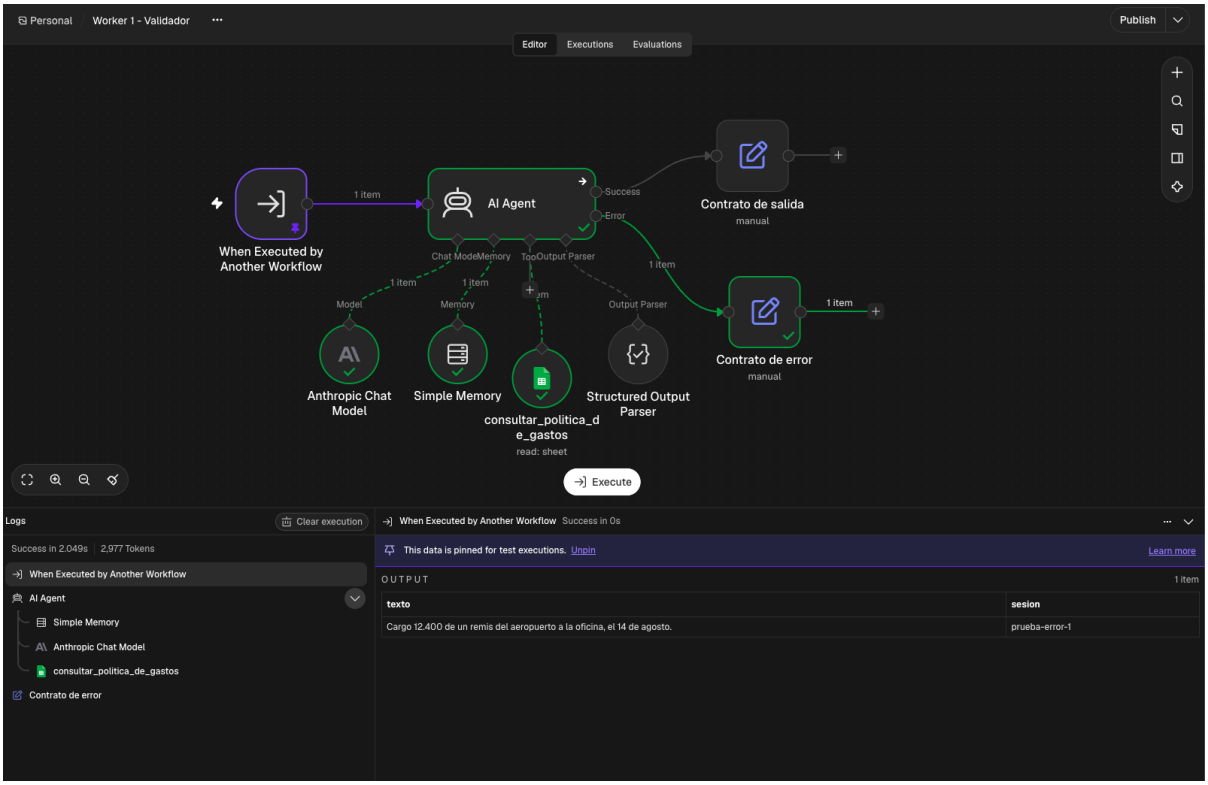


Figura 12. Ruta de error del Worker 1: el agente se corta y sale por Error hacia Contrato de error.

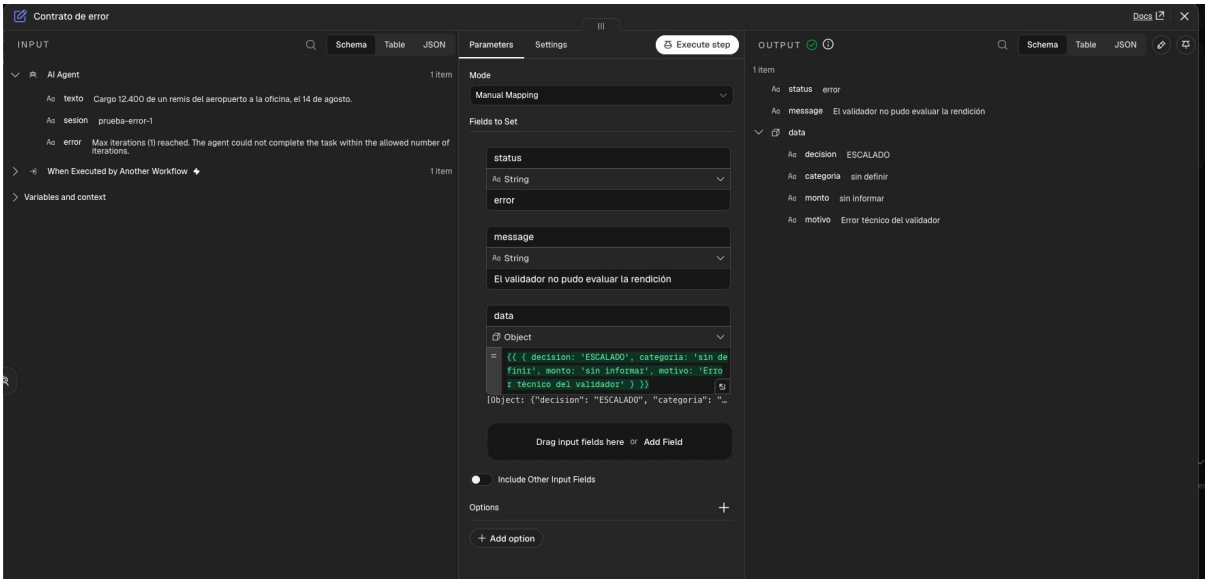


Figura 13. Contrato de error del Worker 1: status "error" y decisión segura ESCALADO.

6.2 Worker 2 · Redactor (forzando un modelo inexistente)

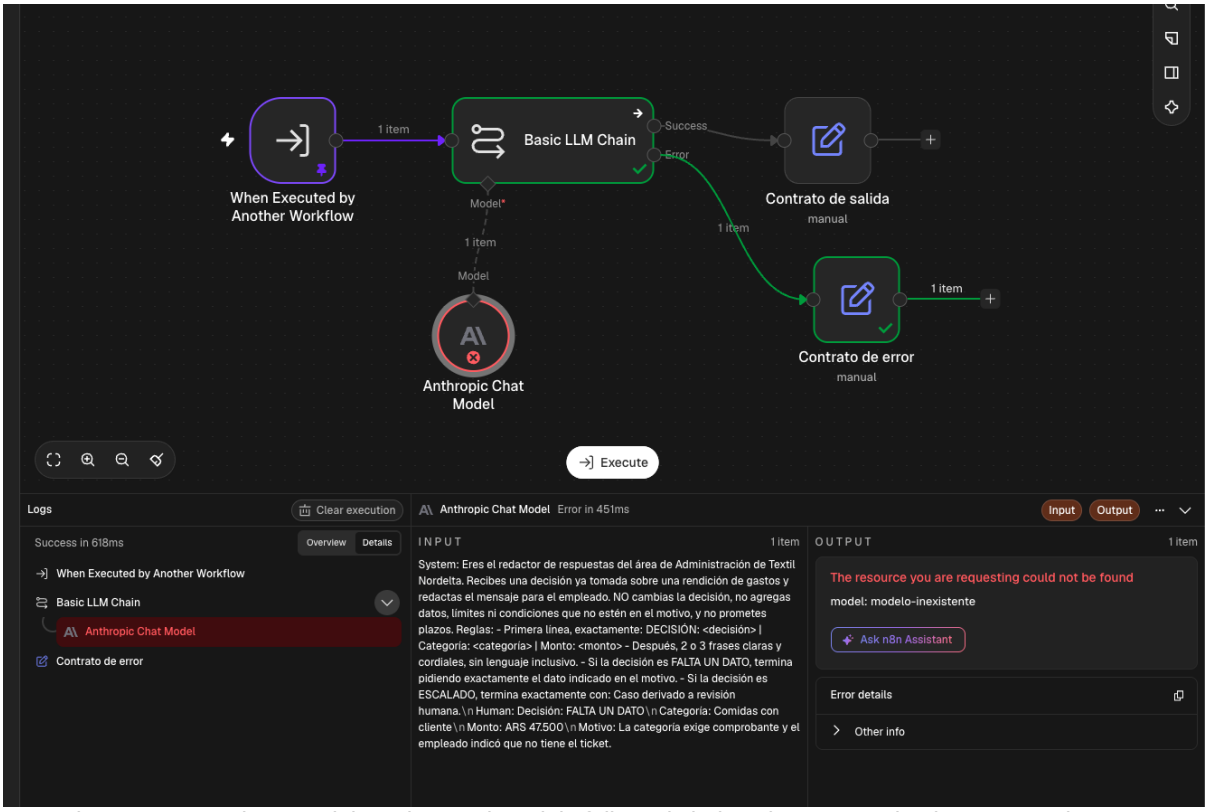


Figura 14. Ruta de error del Worker 2: el modelo falla y el chain sale por Error hacia Contrato de error.

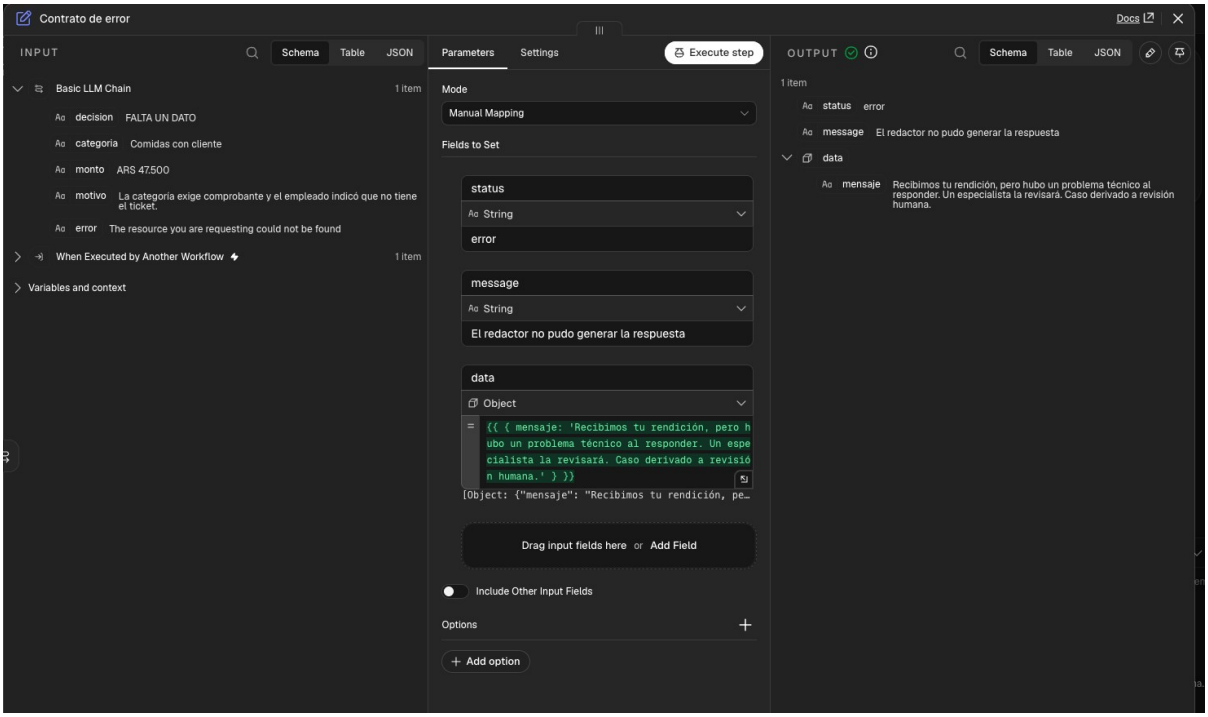


Figura 15. Contrato de error del Worker 2: status "error" y mensaje de contingencia para el empleado.

7. Escapes y respuestas fijas

Lo que no es una rendición no consume Workers: los temas ajenos reciben una respuesta fija y lo que debe resolver una persona se escala a un supervisor.

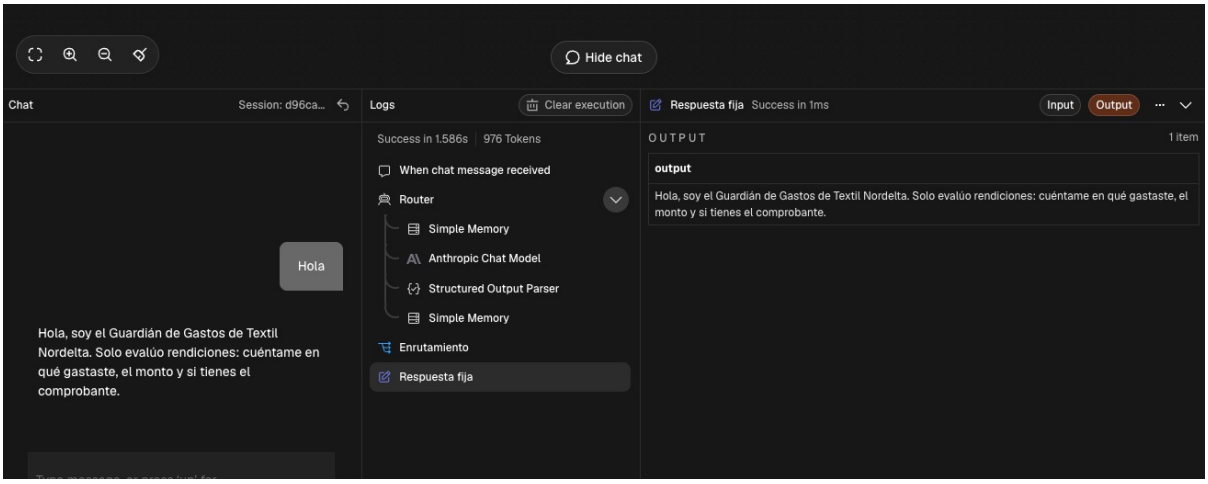


Figura 16. "Hola": rama Otro, respuesta fija sin llamar a ningún Worker.

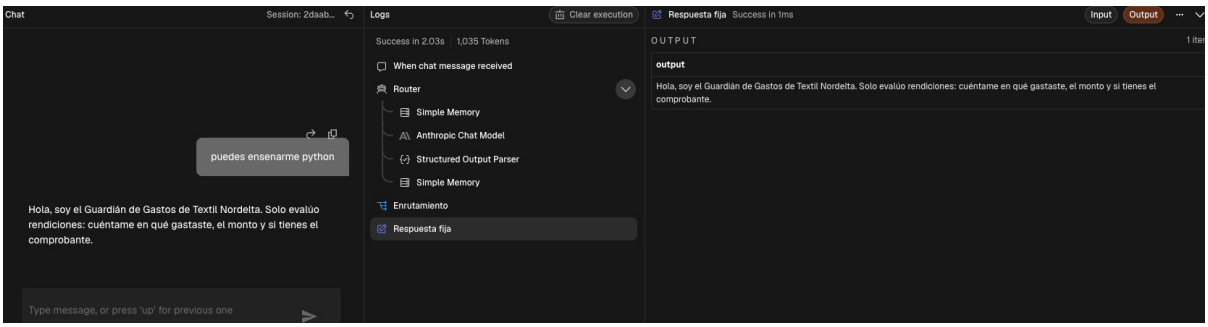


Figura 17. "Puedes enseñarme Python": rama Otro, respuesta fija.

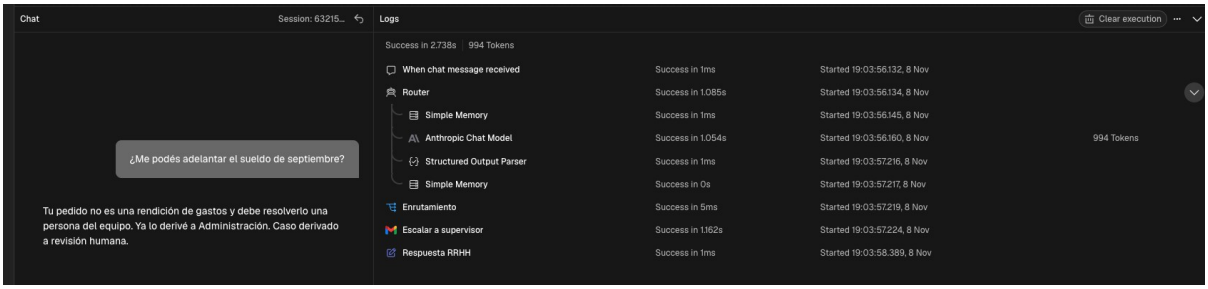


Figura 18. "¿Me podés adelantar el sueldo de septiembre?": rama RRHH, escalamiento y respuesta fija.



Figura 19. Mail de escalamiento recibido por el supervisor.

8. Contrato de datos

Cada handoff pasa por un nodo Edit Fields (Set) que envía solo los campos necesarios, y cada Worker responde siempre con la misma estructura { status, data }, tanto si funciona como si falla.

Tramo	Qué viaja	Formato
Manager → Worker 1	Mensaje del empleado e ID de sesión	`{ "texto": "...", "sesion": "..." }`
Worker 1 → Manager (éxito)	Decisión según la política	`{ "status": "success", "data": { "decision", "categoria", "monto", "motivo" } }`
Worker 1 → Manager (error)	Decisión segura por defecto	`{ "status": "error", "message": "...", "data": { "decision": "ESCALADO", ... } }`
Manager → Worker 2	La decisión ya tomada	`{ "decision", "categoria", "monto", "motivo" }`
Worker 2 → Manager (éxito)	Mensaje para el empleado	`{ "status": "success", "data": { "mensaje": "..." } }`
Worker 2 → Manager (error)	Mensaje de contingencia	`{ "status": "error", "message": "...", "data": { "mensaje": "..." } }`

Los Workers no conocen al Manager: solo reciben campos y devuelven el contrato. Ante un fallo, el Validador devuelve ESCALADO, así un error técnico nunca termina en una aprobación. El Manager llama a ambos con Wait For Sub-Workflow Completion (Wait for child to finish) activado para usar su respuesta en el paso siguiente.

9. Criterio de enrutamiento

El Router es un AI Agent (Claude Haiku 5.5, temperatura 0, Max Iterations 3) con una taxonomía cerrada de tres categorías y un Structured Output Parser que lo obliga a responder { "categoria": "..." }:

- **RENDICION:** el empleado informa un gasto, aunque falten datos → Validador y luego Redactor.
- **RRHH:** sueldos, adelantos, vacaciones o reclamos → mail de escalamiento a un supervisor y respuesta fija.
- **OTRO:** saludos o temas ajenos → respuesta fija, sin llamar a ningún Worker.

Ante la duda, el Router elige RENDICION, porque el Validador sabe pedir el dato que falta. Después del Router, un Switch determinista abre cada rama sin gastar tokens, y su salida de fallback ("Sin clasificar") va al supervisor humano.

La rama de rendición separa responsabilidades: el Validador es el único que ve la política y decide; el Redactor no tiene herramientas y no puede cambiar la decisión, solo redactarla. Además es más eficiente: en las pruebas, el Validador usó unos 6.000 tokens por caso y el Redactor unos 700.